Lab 3: Architecture Justification

Self-Service Coffee Kiosk System | Pradeep | PES1UG25AM809 | Section C

Architecture Selection

We chose **Layered Architecture** for the **Self-Service Coffee Kiosk** System. The kiosk is split into a Presentation Layer (Touch Screen UI), a Business Layer (Order Manager, Payment Service, Receipt Printer) and a Data Layer (Menu & Pricing Database). Each layer talks only to the layer directly below it, through the interfaces IOrder, IPayment, IReceipt and IMenuData.

Two Reasons for the Choice

- **Fixed, small scope with a natural top-down flow.** The kiosk has only 3 coffee types, 2 sizes, card-only payment and one receipt printer. A customer request always flows UI → order logic → payment / printer / database, which maps directly onto layers. Microservices would add network hops, service discovery and deployment overhead that a single-device kiosk does not need, and client-server would be wrong because the kiosk is a standalone unit with no multi-client load.
- **Separation of concerns and easy replacement of components.** Hardware-facing parts (touch screen, card payment, receipt printer) and menu data change independently. Because every component hides its internals behind an interface, we can change the printer model, switch payment provider, add a new coffee type or redesign the UI without rewriting the Order Manager. This also keeps unit testing simple, since each layer can be tested with stubs for the layer below.

Security Advantage

The UI layer has no direct access to payment or database components. All requests must pass through the Order Manager, which validates them (valid drink, valid size, valid amount) before anything is processed. Card data is handled only inside the Payment Service, so it is never stored in the database or printed on the receipt beyond the last 4 digits (PCI-DSS style masking). The database is reachable only from the Business Layer, which limits the attack surface and blocks direct SQL injection from the touch screen.

Performance Benefit

All layers run on the same kiosk, so calls between them are in-process method calls with no network latency. Menu and price data is small and can be cached by the Order Manager at start-up, so selecting a drink and size needs no database round-trip. This keeps screen response and order confirmation fast during busy café hours, and the layers keep independent work simple: payment authorisation does not block the UI thread, and printing runs after payment succeeds.

Trade-off Acknowledged

Layering adds a small overhead because each request passes through several layers, and a failure in the Order Manager affects the whole kiosk. For this scale, the overhead is negligible compared to the maintainability and security gains.